TUGAS 02

Bab 2 — Konsep Container dan Instalasi Docker

Matakuliah Workshop DevOps

Dosen Pengampu : Bapak Dr.Ferry Astika Saputra,ST, M.Sc



Oleh :

Nama : Toimul Setyo Andri

NRP : 3126640021

Kelas : D4 LJ TI kelas B

Politeknik Elektronika Negeri Surbaya

Departemen Teknik Informatika dan Komputer

Program Studi Teknik Informatika

September 2026

Evaluasi dan Latihan Mandiri

1. Mengapa penggunaan tag latest tidak dianjurkan untuk deployment yang harus reproducible?

Karena *build* atau *restart* container di masa depan bisa menggunakan versi image baru yang memiliki perubahan dependensi, fitur, atau pembaruan yang dapat merusak aplikasi (*breaking changes*). Sebaiknya menggunakan *specific version tag* (seperti 1.26-alpine) atau SHA-256 *digest* untuk kepastian lingkungan.

2. Peran containerd dan runc dalam arsitektur Docker

- **containerd**: *High-level container runtime* yang bertugas mengelola siklus hidup container secara penuh (transfer/pull image, manajemen penyimpanan, eksekusi container, hingga jaringan).
- **runc**: *Low-level container runtime* berbasis standar OCI (*Open Container Initiative*) yang bertugas langsung melakukan *spawning* dan menjalankan proses container dengan memanfaatkan fitur kernel Linux (seperti *cgroups* dan *namespaces*).

3. Konsekuensi keamanan dari memasukkan user ke group docker

Grup docker memiliki akses langsung ke Docker Daemon socket (/var/run/docker.sock) yang berjalan sebagai root. User di dalam grup ini dapat mengeksekusi container dengan *volume mount* direktori *root* host (-v /:/host) dan memperoleh akses *root* penuh pada sistem operasi host, hal itu sama saja dengan memberikan hak akses sudo tanpa kata sandi.

4. Perbandingan layer image nginx:1.26-alpine vs image custom

- **Image Base (nginx:1.26-alpine)**: Menggunakan distro Alpine Linux yang sangat ringan, terdiri dari layer sistem operasi dasar Alpine dan instalasi biner Nginx.
- **Image Custom (pens-web:1.0)**: Dibangun di atas base image nginx:1.26-alpine dengan tambahan layer baru dari instruksi LABEL, COPY index.html, dan pengaturan instruksi eksekusi CMD. Image custom memiliki layer aplikasi tambahan yang berisi file HTML spesifik milik pengguna.

5. Kapan sebaiknya memilih VM daripada container?

Menggunakan Virtual Machine (VM) apabila:

- membutuhkan isolasi keamanan tingkat keras (*hardware-level isolation*),
- menjalankan sistem operasi yang berbeda dari OS host (seperti Windows di atas Linux host),
- menjalankan aplikasi *legacy* yang memerlukan akses penuh ke kernel terpisah.

Laporan Praktikum

1. Bukti Minimum

1.1. Screenshoot docker compose ps atau docker ps yang menunjukkan service berjalan.

```
toimul@DESKTOP-2OIAP1F:~/docker-lab/custom-web$ docker run -d --
name pens-app -p 9090:80 pens-web:1.0
2e25d14076673d2d2b8ff985cde2ca03d0b572045ebfcf162802513ae9a4cacb
```

1.2. Screenshot hasil curl/browser/API/dashboard sesuai target bab.

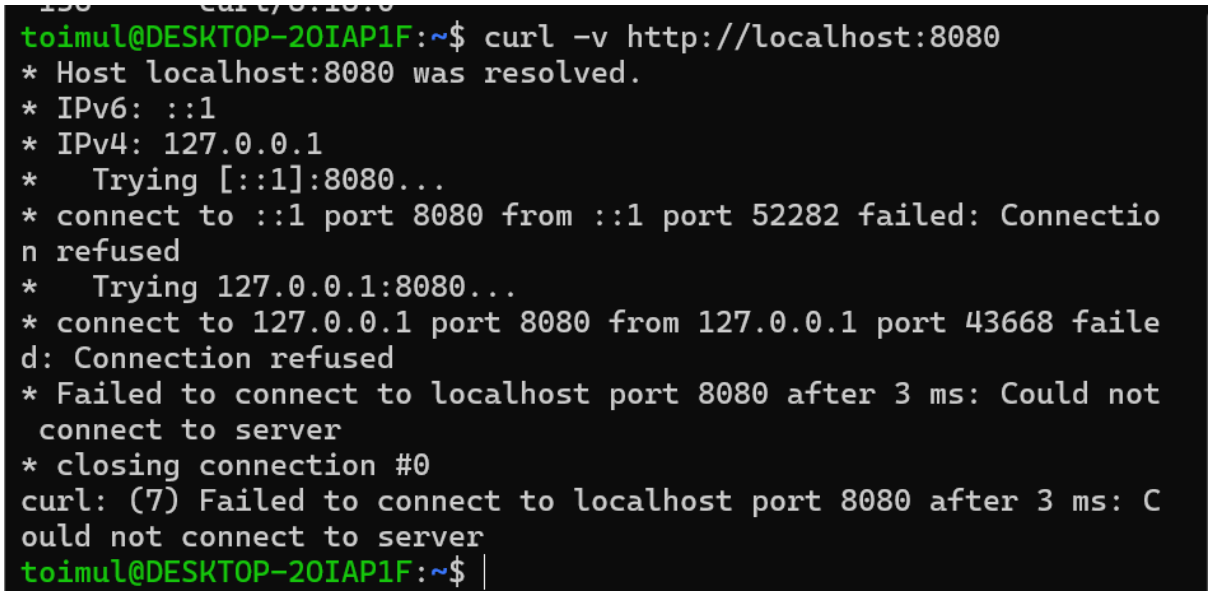
```
toimul@DESKTOP-2OIAP1F:~/docker-lab/custom-web$ curl http://loca
lhost:9090
<h1>Docker Lab PENS</h1>
<p> Nama Mahasiswa: Toimul Setyo Andri<p>
<p>NRP:3126640021</p></p>
<p>ket: Container berhasil berjalan.</p>
```

1.3. Cuplikan log atau query yang membuktikan sistem bekerja.

```
toimul@DESKTOP-2OIAP1F:~$ docker logs --tail 20 web-public
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /
etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in
 /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-re
solvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubs
t-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-wo
rker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start u
p
2026/09/08 04:17:26 [notice] 1#1: using the "epoll" event method
2026/09/08 04:17:26 [notice] 1#1: nginx/1.26.3
2026/09/08 04:17:26 [notice] 1#1: built by gcc 12.2.0 (Debian 12
.2.0-14)
2026/09/08 04:17:26 [notice] 1#1: OS: Linux 6.18.33.2-microsoft-
standard-WSL2
2026/09/08 04:17:26 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024
:1048576
2026/09/08 04:17:26 [notice] 1#1: start worker processes
2026/09/08 04:17:26 [notice] 1#1: start worker process 29
2026/09/08 04:17:26 [notice] 1#1: start worker process 30
```

2. Analisis wajib:

2.1. Jelaskan satu masalah yang muncul dan cara Anda mendiagnosisnya.

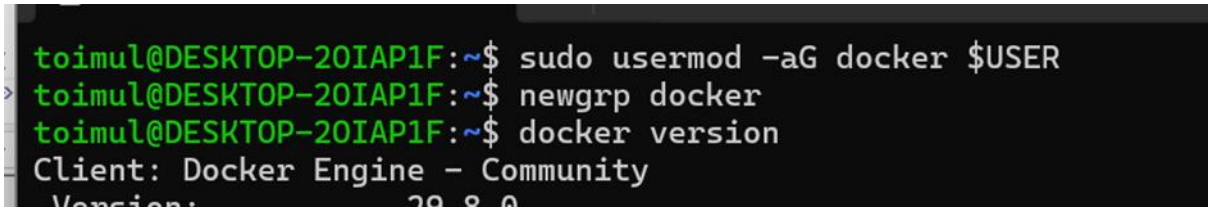


Masalah yang Muncul: Permintaan HTTP ke http://localhost:8080 mengalami kegagalan koneksi dengan pesan kesalahan curl: (7) Failed to connect to localhost port 8080

Dianogsis:

1) Memeriksa daftar container aktif dengan docker ps, ditemukan bahwa container web-public telah dihapus pada langkah pembersihan *docker rm -f* .
2) Container yang sedang berjalan saat ini adalah pens-app yang dipetakan ke port 9090
3) Kegagalan koneksi ke port 8080 terjadi karena tidak ada service/container yang mendengarkan pada port tersebut, sehingga permintaan harus diarahkan ke port 9090.

2.2. Jelaskan risiko keamanan atau operasional yang relevan pada bab ini.



Resiko keamanan:

Menambahkan user toimul ke grup docker, memberikan akses penuh ke socket Docker Daemon. User non-root tersebut dapat mengeksekusi container dengan bind mount ke sistem file host dan memperoleh hak akses root penuh pada sistem host.

Risiko Operasional:

Menjalankan container dengan perintah docker run manual tanpa batas memori/CPU (resource limits) berpotensi menyebabkan satu container mengonsumsi seluruh sumber daya host (noisy neighbor problem).

2.3 Berikan rekomendasi perbaikan bila lab ini akan dibawa ke production-like environment.

1) **Implementasi Rootless Docker Mode**: Menjalankan Docker Daemon dalam mode *rootless* untuk mencegah eskalasi akses ke host jika terjadi *container breakout*.
2) **Konfigurasi Non-Root User pada Dockerfile**: Mengubah Dockerfile agar tidak menggunakan user root saat menjalankan Nginx, dengan menambahkan direktif USER nginx atau *unprivileged user*.
3) **Penerapan Tagging & SHA-256 Digest**: Mengikat image dasar dengan *digest SHA-256* alih-alih hanya nama tag versi untuk menjamin imutabilitas dan *reproducibility* build.